НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №6. Responsive Layout, Container Queries, Fluid Typography та debugging CSS Layout

з дисципліни «Сучасна веброзробка: HTML, CSS, JS/TS»

1. Мета роботи

Закріпити практичні навички побудови адаптивних вебінтерфейсів за допомогою сучасних CSS Layout-механізмів та навчитися знаходити й виправляти проблеми responsive layout за допомогою Browser DevTools.

У межах роботи студент поступово реалізує адаптивний інтерфейс, використовуючи:

- Flexbox;
- CSS Grid;
- Media Queries;
- Subgrid;
- Container Queries;
- Fluid Typography з `clamp()`;
- CSS logical properties та відносні одиниці;
- інструменти Browser DevTools для аналізу layout.

Особливу увагу приділено не написанню CSS «за шаблоном», а експериментуванню, спостереженню за поведінкою layout та аргументованому виправленню проблем.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- розуміти різницю між одновимірним Flexbox та двовимірним CSS Grid;
- уміти комбінувати Flexbox і Grid в одному інтерфейсі;
- уміти створювати responsive layout без надмірної кількості breakpoint-ів;
- розуміти призначення Media Queries та випадки, коли вони не потрібні;
- уміти використовувати `repeat()`, `minmax()`, `auto-fit` / `auto-fill`;

- розуміти призначення subgrid та використовувати його для вирівнювання вкладених компонентів;
- розуміти різницю між Media Queries і Container Queries;
- уміти створювати компонент, який адаптується до ширини свого контейнера, а не viewport;
- використовувати `clamp()` для fluid typography;
- використовувати Browser DevTools для аналізу Grid/Flexbox, computed styles, box model та overflow;
- знаходити причину responsive-проблеми, а не лише усувати її візуальний прояв;
- критично перевіряти CSS-код, запропонований AI-агентом.

3. Завдання лабораторної роботи

1. Провести короткі дослідження Flexbox, Grid, Media Queries, Container Queries та Fluid Typography.
2. Виконати серію code challenges зі зростаючою складністю.
3. Побудувати адаптивний layout без використання фіксованих ширин там, де вони не потрібні.
4. Реалізувати компонент, який використовує Container Query.
5. Реалізувати fluid typography без набору breakpoint-ів для кожного розміру viewport.
6. Використати Browser DevTools для пошуку та виправлення навмисно закладених layout-проблем.
7. Використати AI-агента для аналізу щонайменше одного challenge та перевірити запропоноване рішення самостійно.
8. Підготувати лист із посиланнями на деплой та висновком за результатами експериментів і прийнятими CSS-рішеннями (§8).

> **Starter-проєкт.** Розмітка для всіх challenges і сторінка з навмисно закладеними помилками для Challenge 8 — layout-lab-starter.zip. Опис вмісту — у `README.md` всередині архіву.

4. Дослідницькі завдання

Перед виконанням практичної частини опрацюйте теоретичні матеріали та проведіть короткі експерименти. Результати експериментів (спостереження та висновки) знадобляться для висновку в листі (§8) та усного захисту (§10). Загальна довідка — Learn CSS — web.dev та CSS — MDN.

4.1. Flexbox та Grid

Опрацювати:

- Layout — web.dev Learn CSS
- Flexible box layout — MDN
- CSS grid layout — MDN

Дослідити:

- Яку задачу вирішує Flexbox?
- Яку задачу вирішує CSS Grid?
- Чому Flexbox називають one-dimensional layout, а Grid — two-dimensional layout?
- Що змінюється після застосування `display: flex`?
- Що змінюється після застосування `display: grid`?
- Як працюють `flex-wrap`, `justify-content`, `align-items`?
- Як працюють `minmax()`, `repeat()` та одиниця `fr`?

Мініексперимент

Створіть контейнер із шістьма елементами та послідовно перевірте:

```
display: flex;
display: grid;
grid-template-columns: repeat(3, 1fr);
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Зафіксуйте, як змінюється поведінка layout при зміні ширини viewport.

4.2. Media Queries

Опрацювати:

- Media queries — MDN

Дослідити:

- що саме визначає `@media`;
- чим Media Query відрізняється від простої зміни CSS-властивості;
- які breakpoint-и дійсно потрібні;
- чи можна вирішити задачу за допомогою Grid/Flexbox без Media Query;
- як перевіряти Media Queries у DevTools.

Експеримент

Порівняйте два підходи:

```
@media (width < 768px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}
```

Сформулюйте висновок: коли Media Query справді потрібна, а коли responsive-поведінка може бути властивістю самого layout.

4.3. Container Queries

Опрацювати:

- Container queries — MDN
- CSS containment — MDN

Дослідити:

- чим `@container` відрізняється від `@media`;
- для чого потрібен `container-type`;
- що означає `container-type: inline-size`;
- як визначається контейнер, який перевіряється;
- у яких випадках Container Query робить компонент більш reusable.

Базовий експеримент

```
.card-wrapper {
  container-type: inline-size;
}

@container (width > 500px) {
  .card {
    /* змінити layout */
  }
}
```

Змініть ширину саме `.card-wrapper`, не змінюючи viewport, та спостерігайте за результатом.

4.4. Fluid Typography

Опрацювати:

- Adapting typography to user preferences with CSS — web.dev
- clamp() — MDN

Дослідити:

- різницю між px, rem, vw;
- призначення `clamp()`;
- структуру `clamp(min, preferred, max)`;
- чому fluid typography може зменшити кількість breakpoint-ів.

Експеримент

Порівняйте:

```
h1 {
  font-size: 32px;
}

@media (width >= 768px) {
  h1 {
    font-size: 48px;
  }
}
```

```
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
}
```

Змініть ширину viewport та за допомогою DevTools перевірте computed `font-size`.

4.5. Subgrid

Опрацювати:

- CSS subgrid — web.dev
- Subgrid — MDN

Дослідити:

- яку проблему вирішує subgrid;
- чим nested grid відрізняється від subgrid;
- як успадковуються tracks батьківського Grid;
- у яких випадках subgrid допомагає вирівняти вміст карток.

Експеримент

Обидва варіанти — картки всередині спільного Grid-контейнера:

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 24px;
}
```

Nested Grid — кожна картка має власні рядки:

```
.card {
  display: grid;
  grid-template-rows: auto 1fr auto;
}
```

Subgrid — картка займає три рядки батьківського Grid і використовує саме їх:

```
.card {
  display: grid;
  grid-row: span 3;
  grid-template-rows: subgrid;
}
```

Без `grid-row: span 3` картка займає лише один рядок батька, і `subgrid` не має чого успадковувати.

Поясніть: яку проблему вирішує subgrid, яку складніше вирішити звичайним nested Grid?

5. Практична частина: Code Challenges

Розмітка до кожного challenge є у starter-проєкті.

Challenge 1. Flexbox: responsive header

Початковий стан

```
<header class="header">
  <a class="logo" href="#">WebLab</a>

  <nav class="navigation">
    <a href="#">Home</a>
    <a href="#">Projects</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>

  <button class="header-action">Get started</button>
</header>
```

Завдання:

1. розташувати logo, navigation та button в одному рядку;
2. забезпечити вертикальне вирівнювання;
3. забезпечити відстань між елементами через gap;
4. дозволити navigation переноситися на новий рядок;
5. не використовувати `position: absolute` для побудови layout;
6. перевірити поведінку при вузькому viewport.

Додатковий challenge: зробити так, щоб при недостатній ширині navigation автоматично переходила на наступний рядок без Media Query.

Очікуваний результат: header залишається працездатним при різній ширині viewport.

Challenge 2. Grid: adaptive card layout

```
<section class="features">
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
  <article class="feature-card">...</article>
</section>
```

Завдання:

- картки мають мінімальну ширину 220px;
- кількість колонок залежить від доступного простору;
- між картками є однаковий gap;
- немає горизонтального overflow;

- не використовуються окремі Media Queries для 2, 3 та 4 колонок.

Використати: `repeat()`, `minmax()`, `auto-fit`.

Експеримент: порівняти `auto-fit` і `auto-fill` та пояснити різницю за результатом.

```
repeat(auto-fit, minmax(220px, 1fr))
repeat(auto-fill, minmax(220px, 1fr))
```

Challenge 3. Responsive page layout: Grid + Flexbox

Побудувати повний layout.

Desktop:

```
+--------------------------------------+
|                Header                |
+-------------+------------------------+
|             |                        |
|   Sidebar   |           Main         |
|             |                        |
+-------------+------------------------+
|                Footer                |
+--------------------------------------+
```

Grid використати для загального page layout та sidebar + main. Flexbox — для header, navigation, групування action buttons і окремих компонентів.

На вузькому viewport:

```
+--------------------+
|       Header       |
+--------------------+
|        Main        |
+--------------------+
|      Sidebar       |
+--------------------+
|       Footer       |
+--------------------+
```

Студент самостійно визначає breakpoint або, якщо це можливо, реалізує зміну layout без Media Query.

Не використовувати: `float`; таблиці для layout; абсолютне позиціонування як основний layout-механізм; фіксовану ширину всієї сторінки.

Challenge 4. Subgrid: aligned cards

Є картки різної довжини. Потрібно вирівняти заголовки, основний текст і кнопки Read more на однакових рівнях без фіксованих висот.

Спочатку реалізувати рішення без subgrid. Після цього замінити реалізацію на subgrid (див. експеримент у §4.5) і порівняти результат: відповідь на питання §4.5 має спиратися саме на цей challenge.

Challenge 5. Container Query: reusable Card

```
<div class="content-layout">
  <aside class="sidebar">
    <div class="card-wrapper">
      <article class="product-card">...</article>
    </div>
  </aside>

  <main>
    <div class="card-wrapper">
      <article class="product-card">...</article>
    </div>
  </main>
</div>
```

Одна й та сама `.product-card` повинна поводитися по-різному залежно від ширини контейнера: у вузькому контейнері — вертикально, у широкому — в один ряд із зображенням та текстовим блоком.

Умова: не використовувати `@media` для зміни layout картки. Використати `container-type` та `@container`.

```
.card-wrapper {
  container-type: inline-size;
}

@container (width > 500px) {
  .product-card {
    display: grid;
    grid-template-columns: auto 1fr;
  }
}
```

`@container` змінює стилі **нащадків** контейнера, а не самого контейнера, тому `container-type` задається обгортці `.card-wrapper`, а не `.product-card`.

Експеримент: помістити ту саму картку у sidebar, main і modal-like container; перевірити, чи правильно вона адаптується незалежно від ширини viewport.

Додатково: дослідити `container-name` і реалізувати іменований container.

```
container: card / inline-size;
```

Challenge 6. Container Query units

Продовжити попередній challenge та зробити розмір заголовка картки залежним від ширини контейнера.

Дослідити `cqw`, `cqi`, `cqmin`, `cqmax` і порівняти їх із `vw`.

Експеримент: помістити однакову картку у wide container і narrow container, не змінюючи viewport. Пояснити, чому `vw` і `cqw` дають різну поведінку.

Challenge 7. Fluid Typography

```
<section class="hero">
  <p class="eyebrow">Modern Web Development</p>
  <h1>Build interfaces that adapt to their environment.</h1>
  <p class="hero-text">
    Responsive interfaces should adapt naturally to available space.
  </p>
  <a class="button" href="#">Explore</a>
</section>
```

Початковий CSS використовує breakpoint-based typography:

```
.hero h1 { font-size: 32px; }
.hero-text { font-size: 16px; }

@media (width >= 768px) {
  .hero h1 { font-size: 44px; }
  .hero-text { font-size: 18px; }
}

@media (width >= 1200px) {
  .hero h1 { font-size: 56px; }
  .hero-text { font-size: 20px; }
}
```

Замініть його на fluid typography через `clamp()`. Конкретні значення підберіть самостійно.

```
.hero h1 {
  font-size: clamp(...);
}
```

Typography повинна не ставати надто малою, не ставати надто великою та плавно змінюватися без окремого breakpoint для кожного розміру тексту.

DevTools experiment: змінювати ширину viewport, вибрати `h1`, відкрити Computed, перевірити фактичний `font-size` та зафіксувати щонайменше три значення.

Challenge 8. Debug the Layout

Starter-проєкт (тека `challenge-08`) містить навмисно закладені CSS-помилки. На вузькому viewport сторінка має horizontal scroll, обрізаний hero heading, картки виходять за межі контейнера, sidebar займає непропорційно багато місця.

```
.page {
  width: 1200px;
  display: grid;
  grid-template-columns: 300px 1fr;
}

.main {
  width: 1000px;
}

.cards {
  display: flex;
  gap: 24px;
}

.card {
  width: 350px;
}

.hero h1 {
  font-size: 64px;
  white-space: nowrap;
}
```

Завдання:

1. відкрити Browser DevTools;
2. визначити елемент, який створює overflow;
3. перевірити його Box Model;
4. перевірити computed `width`, `min-width`, `max-width`;
5. перевірити Grid/Flexbox overlay;
6. знайти CSS rule, яка створює проблему;
7. визначити першопричину;
8. внести мінімальну зміну;
9. повторно перевірити layout.

Для кожної знайденої проблеми зафіксувати:

Проблема	Причина	CSS-властивість	Виправлення
Horizontal overflow	...	...	...
Heading overflow	...	...	...

Проблема	Причина	CSS-властивість	Виправлення
Card overflow	...	...	...

> **Важливо!** Не вважати `overflow-x: hidden` автоматичним правильним рішенням. Студент повинен пояснити, чому виник overflow, а не просто приховати його.

Challenge 9. AI-generated Layout Review

AI-агент тут виступає у двох ролях: спочатку він **генерує** рішення, потім — **перевіряє** його як reviewer за prompt-ом із §7. Між цими етапами ви перевіряєте код самостійно, щоб мати з чим порівнювати рекомендації AI.

Етап 1. Генерація

Згенеруйте або отримайте від AI-агента CSS-рішення для такого завдання:

> Create a responsive dashboard with a header, sidebar, main content area and a grid of cards. It should work on desktop, tablet and mobile.

Збережіть отримане рішення без змін — наприкінці його треба порівняти з фінальним.

Етап 2. Власна перевірка

1. перевірити AI-generated implementation у Browser DevTools;
2. самостійно знайти щонайменше 3 потенційні проблеми або зайві рішення.

Особливу увагу звернути на надмірну кількість Media Queries, фіксовані ширини, `min-width`, overflow, дублювання CSS, неправильне використання Flexbox/Grid, можливість використання `minmax()`/`auto-fit` та Container Query.

Етап 3. AI code review

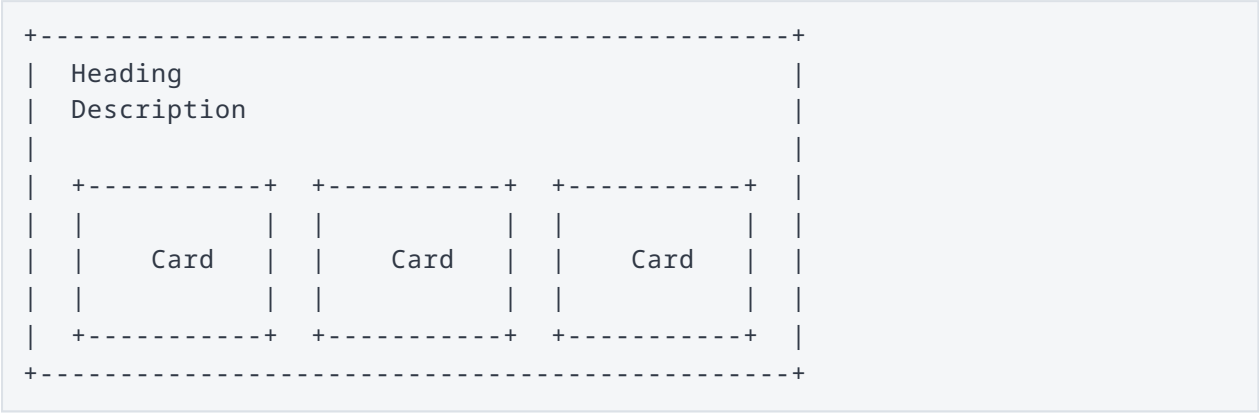
1. передати згенерований код AI-агенту з рекомендованим prompt-ом для code review (§7);
2. перевірити кожну рекомендацію AI за MDN/web.dev і порівняти з результатами власної перевірки: що AI знайшов, а ви — ні, і навпаки;
3. виправити обґрунтовані проблеми;
4. порівняти початковий і фінальний CSS.

AI-рекомендація	Прийнята?	Перевірка	Обґрунтування
...	Так/Ні	MDN/web.dev	...
...	Так/Ні	MDN/web.dev	...
...	Так/Ні	MDN/web.dev	...

Challenge 10. Final Responsive Component

Об'єднати отримані навички в одному невеликому компоненті — Responsive Feature Section.

Desktop:



Mobile:



Обов'язково використати:

- CSS Grid;
- Flexbox щонайменше для одного внутрішнього компонента;
- `minmax()` або `auto-fit`;
- `clamp()`;
- Container Query щонайменше для одного компонента;
- DevTools для перевірки responsive behavior.

Необов'язково: subgrid; container query units; named containers.

6. Вимоги до фінальної реалізації

Фінальна реалізація повинна:

1. коректно працювати на вузькому та широкому viewport;
2. не мати горизонтального overflow без обґрунтованої причини;

3. не містити фіксованої ширини основного page container;
4. використовувати Grid та Flexbox відповідно до задачі;
5. не містити зайвих breakpoint-ів;
6. використовувати fluid typography щонайменше для одного типу тексту;
7. містити щонайменше один Container Query;
8. не використовувати `overflow-x: hidden` як спосіб приховування layout-проблеми;
9. не використовувати абсолютне позиціонування як основний механізм layout;
10. бути перевіреною у Browser DevTools.

7. Вимоги до використання AI

AI-агент дозволяється використовувати для пояснення CSS Layout, генерації початкового рішення, пошуку потенційних проблем, debugging, рефакторингу та порівняння альтернативних реалізацій.

Однак студент повинен самостійно:

- перевірити результат;
- зрозуміти використані CSS-властивості;
- перевірити рекомендації за документацією;
- пояснити причину кожної суттєвої зміни.

Обидва prompts нижче ставлять AI в роль reviewer-а: він аналізує вже написаний код і не змінює його. Prompt для debugging стане в пригоді, зокрема, у Challenge 8; prompt для code review — обов'язковий етап Challenge 9.

Рекомендований prompt для debugging:

> Проаналізуй responsive layout цього компонента. Не змінюй код. Знайди потенційні проблеми з overflow, fixed widths, Grid/Flexbox та Media Queries. Для кожної проблеми поясни першопричину та запропонуй мінімальне виправлення.

Рекомендований prompt для code review:

> Перевір цей CSS layout на відповідність сучасним підходам responsive design. Зверни увагу на Flexbox, Grid, Media Queries, Container Queries, fluid typography, overflow та повторне використання компонентів. Не змінюй код — спочатку сформуй список findings із поясненнями.

8. Вимоги до листа

Тема листа:

```
[Веб] ЛР-6: Responsive Layout, Container Queries, Fluid Typography та
debugging CSS Layout
```

У листі вказати посилання на:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Висновок:

Сформулювати, що нового опановано, які layout-механізми використовувалися, яку проблему вирішив Container Query, яку проблему вирішив `clamp()` та що було знайдено за допомогою DevTools.

9. Оцінювання

Успішний захист — **2 бали** АБО робота на парі.

10. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Навіть якщо частину CSS запропонував AI-агент, ви повинні вміти пояснити кожне рішення, показати його в DevTools і відповісти на питання з демонстрацією екрана.

Орієнтовні питання:

1. У чому принципова різниця між Flexbox та CSS Grid?
2. Що означає one-dimensional layout?
3. Для чого використовуються `flex-wrap`, `justify-content` та `align-items`?
4. Що робить `minmax()`?
5. Чим `auto-fit` відрізняється від `auto-fill`?
6. Коли Media Query дійсно необхідна?
7. Чим `@container` відрізняється від `@media`?
8. Для чого потрібен `container-type`?
9. Що таке named container?
10. Що означають одиниці `cqw` та `cqi`?
11. Для чого використовується `clamp()`?
12. Чому `clamp()` може зменшити кількість breakpoint-ів?
13. Яку проблему вирішує subgrid?

14. Чим nested Grid відрізняється від subgrid?
15. Як Browser DevTools допомагає debug-ити Grid?
16. Як Browser DevTools допомагає debug-ити Flexbox?
17. Що таке horizontal overflow?
18. Чому `overflow-x: hidden` не завжди є правильним виправленням?
19. Як знайти елемент, який створює overflow?
20. Чому фіксовані `width` часто створюють проблеми в responsive layout?
21. Як визначити, чи потрібен новий breakpoint?
22. Як перевірити, що AI запропонував коректне CSS-рішення?
23. Чому компонент із Container Query може бути більш reusable, ніж компонент із Media Query?
24. Які layout-рішення ви б залишили без змін після AI code review і чому?